SafeCharge-RL

Reinforcement learning for EV charging when drivers' departure times are wrong or misreported

Full briefing: everything about the project, from zero. Written so a teammate who has never seen it can follow.

Team 32	Adithyan R (AM.SC.U4AIE23065), Adithya Rajesh (AM.SC.U4AIE23007), Balu M Krishna (AM.SC.U4AIE23026)
Guide	Dr. Geetha M, Dept. of CSE (AI), Amrita Vishwa Vidyapeetham
Prepared	9 Oct 2026, for the mid-review (9-10 Oct). Code: github.com/adithyx-0/safecharge-rl (private repo)

Read this first (one paragraph). Many EVs plug in at a shared charging station. Someone must decide, every 15 minutes, how much power each car gets, without exceeding the station's power limit, while keeping the electricity bill low and getting cars charged before they leave. The scheduler only knows the departure time the **driver typed in**, and drivers often type wrong values, or could type early times on purpose to get charged first. We train an RL policy on real charging data and ask: **how badly does it suffer when reports are wrong, and can a driver cheat it?**

Contents

- 1. The problem in simple words
- 2. Why reinforcement learning
- 3. What others did and the gap
- 4. Our research question and hypotheses
- 5. The data and how we prepared it
- 6. The RL formulation (MDP)
- 7. Algorithm and why
- 8. System design and code map
- 9. Baselines and metrics
- 10. Experiments E1-E6
- 11. Results so far
- 12. What is novel
- 13. Progress, what is left, honest caveats
- 14. Likely questions and answers
- 15. Slide-by-slide plan for the mid-review
- 16. Miscellaneous: glossary, how to run, numbers to remember

1. The problem in simple words

Setting. A workplace car park with N charging ports (54 at Caltech, 52 at JPL). Cars arrive at random times, need some energy (kWh), and leave at some time. The whole car park sits behind one transformer, so the total power of all cars together must never go above a **grid limit**.

Three things pull in different directions:

- **Cost:** electricity price changes through the day (cheap at night, expensive 12:00-18:00), so we prefer to charge when it is cheap.
- **Deadlines:** each driver needs the car charged before leaving, so we cannot just wait for cheap hours.
- **Battery wear:** charging at very high power is harder on the battery. We have no battery data, so we use a simple proxy: how often a port runs above 80% of its maximum power.

The twist (our contribution). The scheduler does not see when the car really leaves. It only sees the departure time the driver *reported*. Real drivers get this wrong (on real data the average error is about 3 hours). A driver could also report an earlier time on purpose, hoping to be served first. The car always leaves at its real time, whatever was reported.

Our research question. When reported departure times are wrong or deliberately early, how much do cost and missed deadlines get worse for a learned policy compared with simple rules, how much extra charge can a cheating driver get, and can a small change in the policy or training reduce that?

Problem statement in one line. Learn a charging policy that never exceeds the grid limit, keeps unmet energy within a small budget (2% of requested energy), keeps battery wear within a budget, and minimises electricity cost, when it only sees reported (possibly wrong) departure times.

2. Why reinforcement learning

- **Decisions are made one step at a time without knowing the future.** Exact optimisation (linear programming) needs all future arrivals, departures and prices. We only use it as an after-the-fact yardstick (the 'offline optimum').
- **Simple rules look at one thing only.** First-Come-First-Served ignores urgency; Earliest-Deadline-First ignores price. A learned policy can weigh price, urgency, fairness and wear together.
- **MPC (re-solving an optimisation every step) is a fair rival,** good when forecasts and reports are accurate, but slower per step and sensitive to wrong inputs. We include it as a baseline.
- **RL has no built-in safety guarantee,** so we add a hard safety layer (section 6) and constraint budgets (Lagrangian).

3. What others did, the gap, and how we differ

Paper	What it does	What is missing
Li, Wan, He (2020), IEEE Trans. Smart Grid	Safe deep RL for charging as a constrained problem	One charger only; no shared grid limit; safety is learned, not guaranteed
Chen, Shi (2022)	Augmented Lagrangian + SAC for EV charging	One EV, synthetic arrivals; no hard-constraint layer
Ting et al. (2024)	Multi-agent RL with a station power limit; early departures	Simulated EVs; departure uncertainty as random noise; no comparison with simple rules
Li, Sun (2023)	Out-of-distribution-aware charging on Caltech data; shows user-entered departure times are unreliable	Small station (2 chargers); no deadlines/prices; not about misreporting
Ferragut et al. (2021)	EDF/LLF with uncertain departures; under-reporting drivers get more service; fix: serve only until the declared departure	Rule-based policies only (no learning), no prices
Gabriele et al. (2026)	Forecast the missing departure time, trained with the controller	Departure missing, not wrong or strategic
Lee, Li, Low (2019)	ACN-Data: the real dataset we use	Not an RL paper

Products: ChargePoint power management (configured limits and schedules), Enel X JuiceNet (closed in N. America Oct 2024), Caltech ACN (uses MPC with driver-entered times), OCPP (a message standard, does not

say how to compute limits). None is a published learned policy.

The gap. To our search, nobody treats the reported departure time as something a driver may deliberately misreport to a *learned* policy, tested on *real* sessions with a *shared hard grid limit*, compared with rules and an offline-optimal schedule. That is what SafeCharge-RL studies. We do **not** claim a new RL algorithm, and we do not claim 'safety projection + Lagrangian' is new by itself.

4. Hypotheses

H1	A policy trained on reported departures gives more charge to drivers who report earlier ones (the Ferragut effect, now for a learned policy).
H2	Cost and missed deadlines get worse as the noise level, or the share of misreporting drivers, grows.
H3	Serving a car only until its declared departure, and randomising reports during training, reduce the misreporting gain, at some efficiency cost.

Limit we state openly: misreporting is simulated, so we make no claim about how often real drivers lie. Data are workplace/campus sites, not home or public charging.

5. The data and how we prepared it

ACN-Data (Caltech's Adaptive Charging Network): real charging sessions, free after registering for an API token. Downloaded and verified: **Caltech 31,424 sessions, JPL 33,638** (equal to the API's own totals, no duplicates, no zero-energy sessions). The token lives in an environment variable / `.env` and is never committed; raw data are gitignored.

Session field	Used as
connectionTime	Arrival time (rounded up to the 15-min grid)
disconnectTime	TRUE departure (rounded down); the car always leaves here
kWhDelivered	Energy the car needs, e_i (clipped to what one port can deliver in the stay)
siteID	Caltech (train/val/test) vs JPL (transfer test)

Constants from the data: each port gives at most **6.656 kW** (32 A x 208 V). Price: SCE time-of-use tariff, summer weekday, $/kWh 0.05623 (00-08h), 0.0925 (08-12h), **0.26668 (12-18h, peak)**, 0.0925 (18-23h), 0.05623 (23-24h).

Choices we made while preparing the data (all in `configs/base.yaml`, logged in `docs/PROJECT_LOG.md`)

Decision	What we did and why
Episode	One weekday = 96 steps of 15 minutes.
Midnight-crossing sessions	Dropped (6.3% at Caltech, 1.5% at JPL). Clipping them would invent an earlier departure, which is exactly what we study.
Caltech fast station	Excluded: one station delivers about 45 kW, not 6.656 kW (3.6% of sessions, 8.6% of energy). Replay shortfall vs recorded energy falls from 7.4% to 1.2%.
Tariff	Summer price curve applied to ALL weekdays (a controlled assumption). Real Jun-Sep days are too few (about 85 in our period) and mixing seasons would add a price shift.
Daylight saving	No weekday affected (clock changes fall on Sundays).
Simulator	Own small numpy/Gymnasium environment (fast, about 32,000 steps/s on one core) instead of ACN-Sim, which works in amps and its data client failed authentication. The design allowed this as its fallback.
Grid cap	P_grid = kappa x P_peak. Chosen kappa = 0.20 (P_grid 34.6 kW): the tightest value where the 2% unmet budget is still achievable with full information. Also tested 0.15 (too tight) and 0.30 (loose). JPL uses 0.28 (95 kW).

Splits (chronological, one pricing regime, before COVID)

Split	Dates (local)	Days	Vehicles
Train	2019-01-01 to 2019-08-31	174	6,206
Validation	2019-09-01 to 2019-10-31	44	1,536
Test	2019-11-01 to 2020-02-29	85	2,720
Post-shift test	2021-01-01 to 2021-09-30 (after COVID)	182	only 9 per day (vs 34)
JPL transfer	same test / post windows	85 / 176	5,329 / 5,533

Surprising finding about reports

The submitted text says only 0.2% of sessions have a driver-entered departure time. **That is wrong for the full data** (the 500-session sample was all from April 2018, before the app collected it): **52% of Caltech and 94% of JPL sessions have one.** Real drivers are far from accurate: mean error 3.15 h (Caltech) and 2.35 h (JPL); only about 35-39% are within 1 hour; about 60% say a time earlier than the real one. We kept the design's modes unchanged and added the recorded reports as an extra mode, **M1r** (supplementary). The team must correct the 0.2% statement in the report.

Mode sizes: the design's noise w up to 2 h gives mean error w/2 = 1 h at most, much milder than reality.

6. The RL formulation (MDP)

Part	Definition (plain words)
Agent	The charging controller of the station.
Environment	The station simulator: ports, vehicles from real sessions, price, grid limit. Built by us in numpy.
Episode / end	One weekday of 96 steps (15 min). Ends after step 96. A car's unmet energy is counted when it actually leaves.
State (4N+4 numbers)	Per port: occupied?, energy still needed r, time left until REPORTED departure tau, laxity = tau - r / P_port (spare time if charged at full rate; negative = cannot finish). Global: price, sin and cos of time of day, last step's total power / grid limit. Empty ports are zero and masked. **The true departure time is never shown.**
Action	A number u between 0 and 1 for each port = the fraction of that port's maximum useful power it asks for.
Reward	Minus the electricity cost of the energy delivered this step (scaled to about unit size).
Cost 1: unmet energy	When a car leaves: energy it did not get / reference energy (8.36 kWh). Budget: 2% of requested energy per day.
Cost 2: battery wear proxy	Share of ports running above 80% of max power at each step. Budget set from the rule-based baselines (2.92 per day).
Safety projection	Hard grid limit: see below. It lives inside the environment step, so the limit can never be broken.
Objective	Maximise reward subject to both cost budgets (a constrained MDP). Discount 0.99 in the design.

The safety projection in one picture

If the policy's requests add up to more than the grid limit, the layer subtracts the **same amount mu** from every request (never below 0) until the total fits. mu is found by a quick bisection (about 20 tries). If the request already fits, nothing changes. This is the exact closest feasible point, and our tests check it against a general QP solver.

Illustrative example (made-up numbers, not data)	
Grid cap	20 kW
Policy asks for (3 cars)	12 kW, 10 kW, 8 kW (total 30 kW, too much)
Safety layer subtracts the same amount mu from each	mu = 3.33 kW
Power actually given	8.67 kW, 6.67 kW, 4.67 kW (total 20 kW, safe)

Reporting modes (how the reported time is made from the true one)

Mode	Meaning
M0	Truthful: reported = true departure.
M2 (noisy)	reported = true + random error uniform in [-w, +w], w = 0.5, 1, 2 hours.
M3 (late)	A share q of drivers report later than the truth by up to w hours.
M4 (strategic)	A share rho (0.1, 0.25, 0.5) report s hours EARLIER, s = 0.5, 1, 2.
M1r (extra)	Real recorded reports from ACN-Data where they exist, else a resampled real error.
Note	There is no 'M1' on purpose (the design dropped it). Reports are never earlier than arrival. All methods get the same random draws.

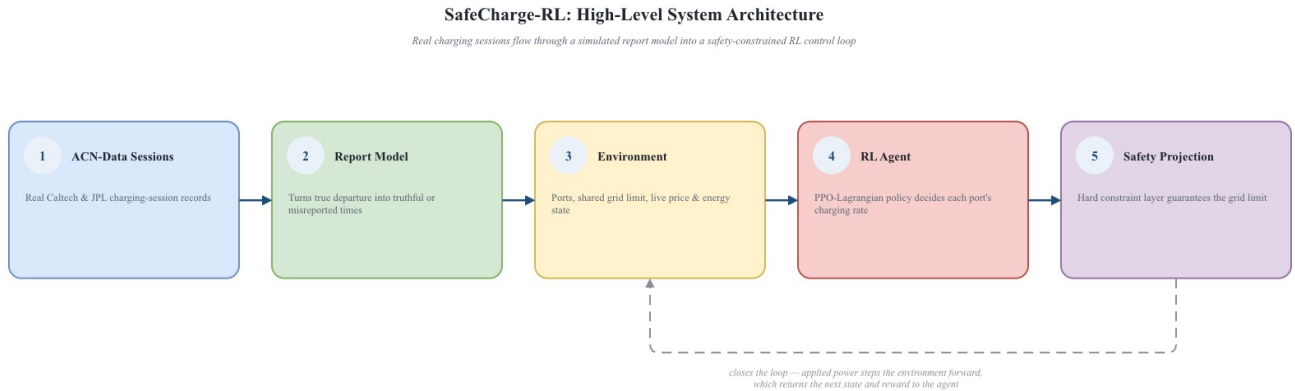
7. Algorithm and why

- **PPO-Lagrangian.** PPO is a standard, stable policy-gradient method. The Lagrangian part adds one multiplier (lambda) per cost; lambda goes up when a cost is over budget and down when under (dual ascent). This turns 'stay within budget' into a trade-off the policy learns. SAC-Lagrangian was 'if time allows' (not done).
- **Policy network: Deep Sets.** One small shared network per port, average over occupied ports to get a 'station context', then a shared head gives each port's request. Sharing parameters means the same policy works for any number of cars and any port order (tested).
- **Critics** (reward, cost 1, cost 2) use the same shared structure.
- **Memoryless:** the observation is treated as the state (true departure is hidden, so this is only approximate).
- **Our own single-file PPO-Lagrangian (about 170 lines)** instead of a library: a library (OmniSafe) was checked but pulls about 40 extra packages and, to our knowledge, supports one cost, while we need two plus a custom network. This differs from the design wording 'existing implementation'.

Additions that are NOT in the submitted design (must be disclosed)

- **Behaviour-cloning warm start from LLF:** from scratch the policy stayed worse than every rule after 2,000 updates (unmet energy arrives only at departure, so learning is slow). We first copy LLF's behaviour, then let RL improve.
- Discount 0.999 instead of 0.99; dual-step clipping and a cap on lambda (lambda once wound up to 89); learning-rate decay.
- Count of occupied ports added to the pooled context (the average alone cannot tell how many cars compete); policy outputs a logit passed through a sigmoid.
- Training: 5 seeds x 1,500 updates each, tuned on validation days only (seed 0); test days used only for reported numbers.

8. System design and code map



Flow: real sessions -> report model (truthful or misreported) -> environment (ports, price, grid limit) -> RL agent proposes power per port -> safety projection enforces the limit -> applied power steps the environment and returns the next state and reward.

File	What it does
src/safecharge/data.py	Loads sessions, local time, 15-min rounding, splits, exclusions

File	What it does
report_model.py	M0, M2, M3, M4 (and M1r) with seeded draws
env.py	Gymnasium environment; projection inside step; optional curtail mode
projection.py	Bisection projection (uniform cut; priority variant for E5)
baselines.py / mpc.py / offline_lp.py	FCFS, EDF, LLF / receding-horizon LP / hindsight-optimal LP
metrics.py	All metrics, bootstrap CIs, paired replay
agents/deepsets.py, ppo_lag.py	The policy network and the PPO-Lagrangian trainer
scripts/	train.py, run_experiment.py, run_paired_replay.py, make_tables.py, data download/audit/calibration
tests/	108 tests, all passing (run on 9 Oct 2026)
results/	CSV tables; FINDINGS_phase1-3.md has the plain bullet findings

9. Baselines and metrics

Baseline	Idea	Sees
FCFS	Serve in arrival order up to the grid limit	ignores reports
EDF	Earliest reported departure first	reported time
LLF	Least laxity first (most urgent)	reported time + energy
MPC	Re-solve an optimisation each step using reported deadlines	reported time
Offline optimum	Linear program with the TRUE departures and the whole day known; a lower bound (checked on every test day)	true time (cheating)
Unprojected RL	RL with a soft grid penalty instead of projection (planned, not built)	-

Metrics: cost per kWh; **unmet energy U** (share of requested energy not delivered); missed-deadline rate (car got under 95% of its request); grid violations (count and size); **high-power ratio W** (battery-wear proxy); Jain fairness; gap to the offline optimum; **misreporting gain**; decision time per step. Mean over test days with 95% bootstrap confidence intervals; identical days and report draws for every method.

Paired replay (the misreporting-gain test). Pick one focal car. Run the same day twice with the same policy and the same other reports: once the car reports truthfully, once it reports s hours earlier. Gain = extra energy it received in the second run (kWh and % of its request). With s = 0 the gain is exactly 0 (tested).

10. Experiments

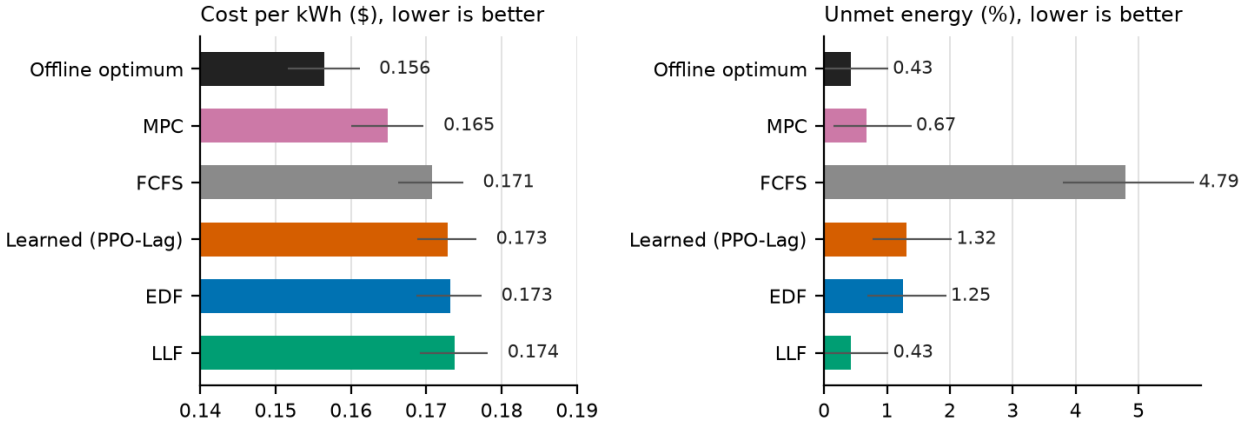
ID	Question	Status
E1	Learned policy vs baselines, truthful reports	Done (5 seeds)
E2	How much worse with noisy (M2) / late (M3) reports?	Done (5 seeds)
E3	Can a driver gain by reporting early? (paired replay, tests H1, H2)	Done (5 seeds)
E4	Fixes: (a) serve only until declared departure; (b) train with randomised reports (tests H3)	(a) done on existing policy; (b) 6 of 9 training runs finished, 3 paused
E5	Projection ablation: uniform cut vs priority-weighted vs none	Switches built; no test-split run
E6	Transfer: JPL site and post-COVID period	Done

11. Results so far

All numbers are on the 85 test days, kappa 0.20, 5 trained seeds, 0 grid violations for every method. Source: `results/tables/` and `results/FINDINGS_phase1-3.md`.

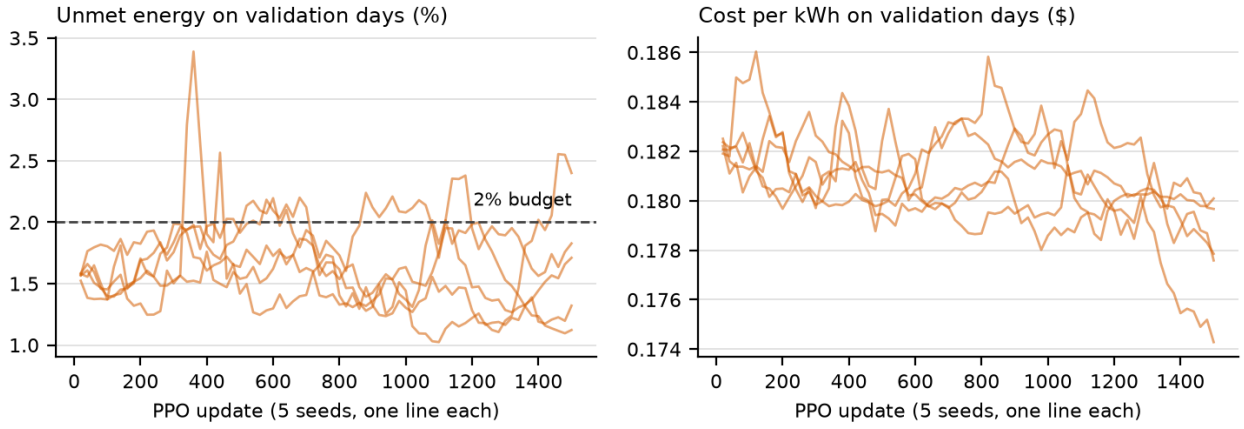
E1: truthful reports

Method	Cost $/kWh	Gap to optimum	Unmet energy	Missed-deadline vehicles	High-power ratio
Offline optimum	0.1564	-	0.43%	0.6%	0.79
MPC	0.1649	+5.5%	0.67%	2.5%	0.77
FCFS	0.1708	+10.2%	4.79%	9.1%	0.78
Learned (PPO-Lag)	0.1728	+11.5%	1.32%	7.4%	0.17
EDF	0.1732	+11.7%	1.25%	1.2%	0.77
LLF	0.1738	+12.1%	0.43%	1.7%	0.77



- The learned policy **ties EDF** on cost and unmet energy. It does **not** beat the rules; MPC (with truthful deadlines) is the cheapest, 5.5% above the optimum.
- Its special trait: **gentle charging.** High-power ratio 0.17 vs 0.77-0.79 for everything else, so the wear proxy is about 40% lower at similar cost.
- Weak spots: 7.4% of cars end below 95% (EDF 1.2%), and short stays (under 2 h) get 3.0% unmet vs 0.0-0.5% for EDF/LLF.
- Decision time per step: rules 0.045 ms, learned 0.14 ms, MPC 0.48 ms.

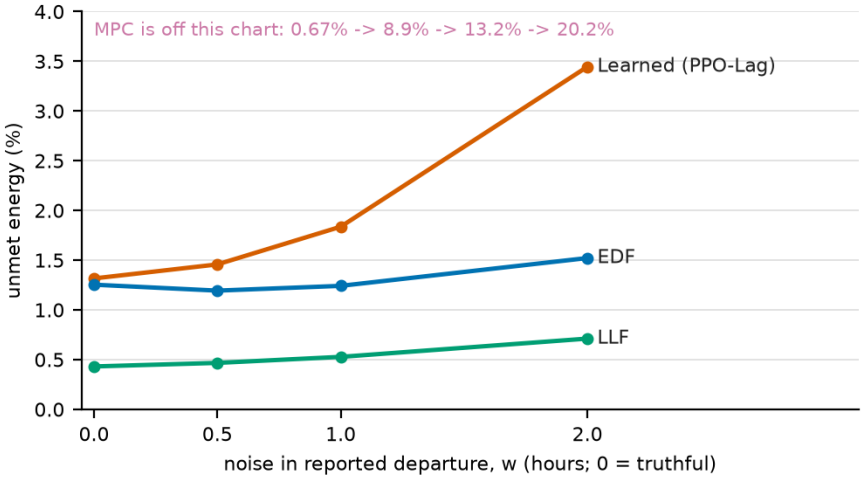
Training curves (validation days)



The policy starts near LLF (warm start) and then stays near the 2% unmet-energy budget while cost settles. It never reaches the price-shifting savings of MPC or the optimum.

E2: wrong reports make things worse

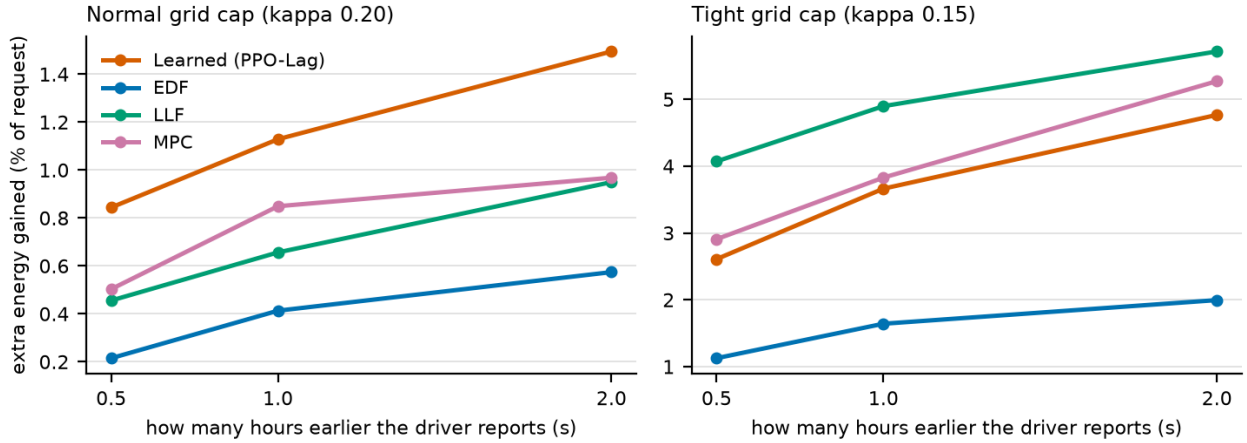
Unmet energy (%)	Learned (PPO-Lag)	EDF	LLF	MPC
Truthful (M0)	1.32	1.25	0.43	0.67
Noisy w=0.5 h	1.46	1.19	0.47	8.86
Noisy w=1 h	1.84	1.24	0.53	13.24
Noisy w=2 h	3.45	1.52	0.71	20.17
Late, 50% of cars (M3)	2.74	1.40	0.56	18.83
Recorded real reports (M1r)	7.40	3.81	3.04	19.56



- Learned policy is about **2.6x worse** at w = 2 h (1.3% to 3.45%); EDF/LLF about 1.2-1.6x; **MPC about 30x worse** (it trusts deadlines and postpones charging). MPC only looks cheap because it delivers less energy.
- With real recorded reports (M1r) the ranking is the same: LLF 3.0%, EDF 3.8%, learned 7.4%, MPC 19.6%.

E3: can a driver cheat? (paired replay, 413 focal cars per cell)

Method	Gain at s=0.5 h	s=1 h	s=2 h	Share of drivers who gain
Learned (PPO-Lag)	+0.84%	+1.13%	+1.49%	93-97%
EDF	+0.21%	+0.41%	+0.57%	1-2%
LLF	+0.46%	+0.66%	+0.95%	2-2%
MPC	+0.50%	+0.85%	+0.97%	3-4%



- **H1 supported:** reporting earlier gives the learned policy's driver extra energy in 93-97% of cases (+0.84% to +1.50% of the request). Rules give a gain only when there is real contention (1.5-3.6% of cases).
- **H2 supported:** gain grows with s and is larger when the cap is tight (learned policy +2.6% to +4.8% at kappa 0.15; LLF is as exploitable as the learned policy there).

- In kWh the gain is small (0.03-0.55 kWh per car) because most cars are not in contention.

E4 part 1: curtail at declared departure (fix tested on existing policy)

- With curtailment **every method loses by under-reporting** (about -0.6% to -1.8% at s up to 1 h, about -9% at s = 2 h): the incentive disappears. With truthful reports nothing changes (as it must).
- Price: honest-but-wrong early reports are punished. Unmet energy for noisy w = 2 h: EDF 1.5% to 6.2%, learned 3.5% to 8.3%. Late reports are not hurt.

E6: transfer

- **Post-COVID Caltech (M0):** fine (unmet 0.31%, rules at most 0.07%).
- **JPL: the learned policy fails to transfer** (unmet 28.6% vs LLF 0.9%), even with a non-binding cap (23.6%). JPL sessions are about 1.7x larger (14.6 vs 8.4 kWh). The reason was not tested directly.

Checks that give the numbers credibility

- Offline-optimum lower bound asserted on every test day for every method and RL seed; 0 grid violations; same days and report draws for all methods; seed spread small (unmet std 0.22 points).

12. What is novel (list for slide 6)

- **Reported departure treated as something a driver can misreport** to a *learned* policy, on real multi-EV sessions with a shared hard grid limit.
- **Misreporting-gain measure by paired replay**, and its use on RL, rules and MPC side by side.
- **Hard safety projection + Lagrangian budgets** for deadlines and wear in one multi-vehicle station (the combination is stated as not claimed new by itself; the evaluation setting is the contribution).
- **Comparison with a hindsight-optimal schedule** on real sessions, with the lower bound verified on every day.
- **Testing the Ferragut fix** (serve only until declared departure) and randomised-report training on a learned policy.
- **Extra finding:** real recorded driver reports are common and badly wrong (mean error 2-3 h); the submitted '0.2%' claim does not hold on full data.

13. Progress, what is left, honest caveats

Progress. In our estimate roughly 80-85% of the planned work is done (not a measured figure). Everything up to E3, E6 and the first part of E4 is finished and reproducible. The mid-review slide asks for '50% progress'; we are beyond that.

Item	State
Data download, audit, preprocessing	Done, verified
Environment, projection, FCFS/EDF/LLF, offline LP, MPC, metrics	Done, 108 tests pass
PPO-Lagrangian, 5 seeds; E1, E2, E3, E6	Done
E4 part 1 (curtail on standard policy)	Done
E4 part 2 (policies trained with random reports / curtail)	6 of 9 runs finished; 3 seed-2 runs paused at update 1240, 1280, 760 of 1500. Resume command is at the end of docs/PROJECT_LOG.md (about 1-2 h)
E5 projection ablation on test days	Not done (switches exist)
Unprojected RL baseline	Not implemented
SAC-Lagrangian	Not started (optional in design)
Report text and slides	Team to write

Caveats to remember

- The learned policy needs a behaviour-cloning warm start that is not in the submitted design. Without it, results are worse than every rule.

- Constants we chose (kappa 0.20, JPL 0.28, cost budget d2, M = $5/kWh in the LP, reward scale, splits, tariff on all weekdays) are not in the submitted text; they are logged in docs/PROJECT_LOG.md.
- Other-kappa rows (0.15, 0.30) are evaluation-only: the policy was trained at 0.20 and not retrained.
- E4 uses 3 seeds (not 5) for time reasons; the randomised-report mixture weights are our choice.
- The M1r mode (recorded reports) is supplementary, not part of the design's E1-E6.
- The 'why' for the JPL failure and for the policy's gentle charging are plausible explanations, not tested.

14. Likely questions and short answers

Question	Answer
Why does the learned policy not beat the rules?	Cost comes mostly from when you charge; rules already charge early and fast. The policy matches EDF and charges more gently. It is a real finding; we report it as is.
Is the grid limit really guaranteed?	Yes, by construction: the projection is inside the environment step. 0 violations in every run; the projection is tested against a QP solver.
Why not just use MPC?	MPC is best when reports are truthful, but collapses (about 30x worse) when they are wrong, and is slower per step.
Why simulate misreporting?	Real reports are noisy but we cannot know who lies on purpose. We show sensitivity and exploitability, not how often drivers lie.
Is the data real?	Yes, ACN-Data sessions from Caltech and JPL. Price and the grid limit are our assumptions (ACN-Data has no price or battery data).
Why is JPL a failure?	Policy trained on Caltech's smaller sessions; it does not transfer. Honest negative result.
How do you know results are not luck?	5 seeds, 85 test days, bootstrap 95% CIs, same days and draws for all methods.

15. Slide-by-slide plan for the mid-review

Slide	Put on it	Where in this PDF
1 Title, team	Title, 3 names, guide, problem area: constrained RL for EV charging under unreliable reports	Cover
2 Problem	Shared station, grid limit, price, deadlines; reports are wrong or gamed; why sequential RL; one-line problem statement	1, 2
3 Existing work	Table of 5-6 papers, their limitation, our gap	3
4 MDP + algorithm + data	State/action/reward/costs/episode; PPO-Lagrangian + Deep Sets, why; data = ACN-Data, our own simulator	5, 6, 7
5 System diagram	The 5-box figure (docs/architecture_diagram.jpg)	8
6 Novel components	The list in section 12	12
7 Demo and next plan	Results chart(s), training curve, status table, remaining work (E4 finish, E5, report)	11, 13

Tip: show one result slide with the E3 chart and one with E1; the E3 chart is the central novelty.

16. Miscellaneous

Glossary

Term	Meaning
EV, kWh, kW	Electric vehicle; energy (kilowatt-hour); power (kilowatt)
Port / P_port	One charging plug; its max power, 6.656 kW
Grid cap P_grid / kappa	Limit for total station power = kappa x typical peak (kappa 0.20 is a tight cap)
Laxity	Spare time = time left minus time needed at full power
Reported vs true departure	What the driver typed vs when the car really leaves
MDP / CMDP	Decision problem with states, actions, rewards / the same with extra cost budgets
PPO	A stable policy-gradient RL algorithm
Lagrangian / dual ascent	Turn budgets into weights (lambda) that rise when a budget is broken
Deep Sets	Network that handles any number of items in any order
Projection	Shrinking a request to the nearest allowed one
FCFS / EDF / LLF	First-come / earliest-deadline / least-laxity first
MPC	Re-solving an optimisation every step over a short horizon
Offline optimum / LP	Best schedule if the whole day is known; solved as a linear program
U, W	Unmet-energy share; high-power ratio (wear proxy)
Curtailment	Stop charging a car after its declared departure time
Bootstrap CI	Confidence range from resampling test days
Seed	Random-number start; 5 seeds = 5 independent trainings

How to run (from the repo root)

- Setup: Python 3.12 venv, `pip install -r requirements.txt`; get your own ACN token, set `ACN_TOKEN` (or `.env`), run `scripts/download_acndata_windowed.py` (the older offset-based downloader kept hitting server 502 errors).
- Tests: `python -m pytest tests -q` (108 should pass).
- Train one seed: `python scripts/train.py configs/e1.yaml --seed 0` (about 35-40 min per run on a laptop; resumable).
- Evaluate: `scripts/run_eval_suite.sh configs/e1.yaml e1_m0`, then `python scripts/make_tables.py e1_m0`.
- Trained checkpoints are not in GitHub; teammates must retrain or get them from the owner's laptop.
- Never commit `.env` or `data/` (both gitignored).

Numbers worth remembering

Caltech / JPL sessions	31,424 / 33,638
Ports	54 / 52 (Caltech fast station excluded)
Port power	6.656 kW
Grid cap (main)	34.6 kW (kappa 0.20)
Step / day	15 min / 96 steps
Peak price	$0.26668/kWh, 12:00-18:00
Unmet budget d1	2% of requested energy
Test days	85 (Caltech, Nov 2019 - Feb 2020)
Best cost (optimum)	$0.1564/kWh; learned $0.1728; MPC $0.1649
Env speed	about 32,000 steps/s; LP about 3 ms per day
Tests	108 passing

Process notes

- Where to read more: `docs/PROJECT_LOG.md` (every decision and test), `results/FINDINGS_phase1-3.md` (plain findings), `CLAUDE.md` (the implementation brief), `docs/final_sections_VIII-XI.md` (submitted design).
- Server problems seen: ACN API returned transient HTTP 502 errors on deep pages; solved by downloading month by month and checking each month against the API total.
- One bug-style incident: a stale run directory was auto-resumed by a new run of the same name; caught from the log and all seeds restarted clean.
- The ACN token was once hard-coded in a notebook; it was removed and is now only read from the environment. Rotate it if the project is ever shared publicly.